

The product-share encoding

A balanced nonlinear secret-sharing that removes the computational-progress diagonal from a gadgetized circuit

2026-07-23

1 The problem: the progress diagonal

The gadgetizer rewrites a small circuit C into a much larger equivalent G whose gates operate on **secret-shared** logical values. The security goal is that G leak nothing about C beyond its input/output behaviour—in particular, that an adversary cannot tell *where in the original computation* any point of G sits.

The standard sharing is a linear XOR pair: a logical value v is carried on two wires c_0, c_1 with $v = c_0 \oplus c_1$. This is the root of a persistent leak. Because the decode is linear, v is a **degree-1 (affine) function of the wires at every snapshot**. The `hmap_affine` tool measures exactly this: for each prefix i of C and prefix j of G , it fits the best GF(2) affine reconstruction of C ’s state- i from G ’s wires at j and reports the holdout error H (0 = perfectly recoverable, 0.5 = hidden). With linear sharing, at the aligned prefix C_i is recoverable from the shares regardless of C_i ’s algebraic complexity in the input, so a low- H **diagonal** runs down the map. That diagonal reveals the temporal layout of the computation, and it **survives every mixing method** tried (ssg, fmix, generation targeting): mixing re-randomises the sharing, but the value stays affine in the new wires, so the diagonal persists at $\rho = 1$.

The measure is read by the **ridge**, never the mean (which saturates near 0.5): *depth* = mean per-row prominence of the low- H valley above its row background; ρ = “is it a monotone diagonal”; *perm-z* = significance vs. a shuffled null.

2 The encoding

Replace the pair decode with a **balanced nonlinear** one. Each logical value carries, on top of its XOR pair, k permanent product mask terms sourced on a frozen band of extra wires:

$$v_i = \underbrace{w_{u(i)} \oplus w_{u'(i)}}_{\text{linear carrier}} \oplus \underbrace{\bigoplus_j \prod_l (w_{p_{jl}} \oplus a_{jl})}_{k \text{ product terms}} \oplus \underbrace{c_i}_{\text{ledger const}}.$$

Each product term is a conjunction of deg band wires with per-factor offsets a_l ; c_i is a ledger constant. The offsets and c are compile-time bits kept in the gadgetizer’s ledger.

3 Why the linear carrier is not optional—the balance obstruction

One might hope to drop the XOR pair and carry v as a **pure** two-wire product $v = (w_p \oplus a)(w_q \oplus b)$. This is impossible to gate.

Claim. No exact reversible gadget can conditionally flip a value whose decode has unequal class sizes—with any ancillae, garbage, or re-encoding.

Argument. A pure 2-wire product decode is unbalanced: over the four (w_p, w_q) states, $v = 1$ has exactly one preimage $(\neg a, \neg b)$ and $v = 0$ has three (a **3:1** imbalance). Consider realising any gate that must conditionally flip a value—e.g. CNOT: $v_t \oplus = v_x$. Exactness forces the gadget F (a reversible map on all wires, ancillae included, with any new ledger afterward) to biject the state-class $\{v_x = \alpha, v_t = \beta\}$ onto $\{v_x = \alpha, v_t = \beta \oplus \alpha\}$. Counting representations and cancelling common factors, the $\alpha = 0$ equations give $R'(0)/R'(1) = R(0)/R(1)$ and the $\alpha = 1$ equations give $R'(0)/R'(1) = R(1)/R(0)$, where R, R' are the target’s rep-counts before/after. Together these force $R(0) = R(1)$: **the target decode must be balanced**. The pure product is 3:1, so no exact CNOT- (or CCNOT-) gadget exists for it; a bijection cannot map a 3-class onto a 1-class. Randomising (a, b) flips *which* side is heavy but never the imbalance itself.

The balanced-implies-affine lemma closes the loop: every nonlinear 2-wire decode is ungateable, so the nonlinearity must come from **more than the value’s own two wires**—here, a linear carrier pair plus product terms over foreign band wires. A second, independent reason the pure product fails: even granting a gadget, its best degree-1 predictor is right on the whole 1-rep side and, optimising over the three 0-reps, still achieves accuracy **5/6**—a degree-1 leak floor of $H \approx 0.17$. The linear carrier symmetrises the classes and removes this floor.

4 The share-native fold-CG—never reconstruct an operand

The predecessor “deferred-mask” design masked values *between* uses but, at each gate, momentarily **un-masked** the operands so a vanilla gate could read the true value (“peek”), then re-masked. Validation showed this does not move the diagonal: the peek re-exposes each gate’s operands at exactly the use-points that form the diagonal. Masking hides *between* uses; the ridge lives *at* uses.

The product-share CG removes the peek. It applies $v_t \oplus = f(\text{controls})$ by **folding f ’s ANF over the operands’ full decodes**: each control value contributes its summand atoms (two carrier literals, the k product-literal sets, and a constant when the ledger constant / control polarity applies); every cross-product of one atom per control operand is emitted as **one conjunction fragment** written into a random carrier of v_t . A single value’s own product term appears only *inside* a fragment’s firing condition, **never materialised on a wire**. Constants and the gate’s complement go to the ledger constant c_t , so the encoding emits no bare X . No wire is ever written except the target’s carriers—**operands stay masked through the gate**.

5 The frozen source band

The mask sources live on a dedicated band appended after the $2n$ carriers. At the input port each band wire is filled with an unbiased data-dependent bit (a random weight- ≥ 2 XOR of the data wires; on the zero slice, $\langle \alpha, x \rangle$) and then **never written again**. Because the sources are frozen, every registered product term is **time-invariant**—invariant under all RG churn and CG traffic. Nothing is ever flushed except the final unshare strip; a mask is one gate, not seven.

6 k versus degree—two different jobs

k (number of terms) and $\deg$ (literals per term) are independent levers.

- **Degree hides algebraically.** A degree- d term is not in the degree- $<d$ GF(2) span, so against the exact-span measure a single degree- d term forces $H = 0.5$ for *every* adversary of degree $< d$, regardless of k .
- k **hides statistically.** Against a best-approximation adversary the terms pile up: k stacked degree-2 masks push the best affine readout error toward $1/2$ as $\frac{1}{2} - 2^{-(k+1)}$. Against the exact-span measure, $k > 1$ is *redundant* for the degree it already blocks.

A single degree- d mask is a *sparser* perturbation (it flips the value only 2^{-d} of the time), so wider masks are individually *weaker* statistically even as they are stronger algebraically—the case for a mixed design (§8).

7 Towers—raising the degree

To hide from a degree- $(d-1)$ adversary you need degree- d masks. The cheapest realisation is a **wider conjunction**: $\deg = d$ uses d band-wire literals, still one MPMCT gate, degree d in the inputs. Each $+1$ of mask degree forces the adversary from degree D to $D+1$, multiplying its regressor count by $\sim W$ (the wire count) and its solve by W^2 : one control literal for us, a factor $\sim W$ for the attacker.

Cost of a tower level is $\sim$ flat in gate count. The number of fold fragments is set by k and the source-gate arity, not the mask degree; widening only adds control literals to existing fragments. (Measured at $n = 16$: $\deg=3, k=3$ is 5944 gates vs. $\deg=2, k=3$ ’s 6161—slightly *fewer*, as wider atoms hit more literal-contradictions.) The real price is **fragment width**: a degree- d mask folded through a w -operand gate yields fragments up to width $d \cdot w$. Degree-3 reaches width 6 (the frozen-DB $m=6$ cliff), degree-4 width 8 (past it, needing scratch decomposition). Towers are nearly free to *build*; the bill is downstream **mixing difficulty** (§10).

8 The mixed design

Because degree and k do different jobs, the production encoding mixes tiers: k base degree-2 terms (statistical strength; each a strong $1/4$ perturbation) *plus* k_{hi} degree-3 tower terms (degree-2 algebraic hiding). The validated setting is $[2, 2, 3, 3]$ —two $\deg=2$ + two $\deg=3$ terms—which kills the degree-1 *and* degree-2 diagonals with margin (a single $\deg=3$ term leaves a boundary residual; two do not).

9 Validation

All heatmaps read by the ridge measure with verified references (endpoint $H \approx 0$). “ALIVE” = clean diagonal ($\rho \approx 1$); “dead” = median-row prominence at the noise floor and $\rho \approx 0$.

$n = 16$ **degree grid** (fixed C , 3 gadget seeds). The plain gadget’s degree-1 diagonal has depth 0.356, ρ 1.0.

encoding	deg-1 depth	deg-1 ρ	deg-2	deg-3
plain ($k = 0$)	0.356	1.00	—	—
deg-2 masks	dead (0.11)	~ 0.5	ALIVE 0.35 , ρ 1.0	—
deg-3 masks	dead (0.11)	~ 0.5	dead (0.13)	ALIVE 0.33 , ρ 1.0

deg = d masks kill every adversary of degree $< d$; the diagonal returns at degree exactly d (the wall moves up one level per +1 mask degree, it does not vanish). $k = 2$ hides the degree-2 diagonal as completely as $k = 3$. All nulls are capability-controlled: a band-restricted degree-2 adversary *does* recover degree-2 masks (ρ 1.0).

$n = 128$ **sliced sandwich** (real production circuit). Original $C = 3000$ g57 gates; the mixed $[2, 2, 3, 3]$ gadget is 374478 **gates on 576 wires** ($125 \times C$).

gadget	adversary	depth	depthMed	ρ	meanH
plain	degree 1	0.361	0.367	1.00	0.471
mixed $[2, 2, 3, 3]$	degree 1	0.060	0.000	-0.02	0.498
mixed $[2, 2, 3, 3]$	degree 2	0.060	0.000	-0.04	0.498

The mixed encoding kills both diagonals on the real sandwich; the median original-circuit prefix has zero ridge prominence. Control: the plain gadget (512 wires) vs. mixed (576, only +64 band) reads a perfect diagonal, so the mixed flatness is real hiding, not a wire-count artifact.

10 Interaction with mixing—a caveat and a feature

The wide mask fragments (width up to ~ 15 after folding through the 256-wire sandwich’s CC-NOTs) are **ineligible for frozen-DB re-encoding** (the narrow $m1$ – $m11$ store, `--db-ctrl-cap 2`). Operationally, **generation-targeting mixing does not converge** on a product-share gadget: the 5th-percentile gate generation is pinned at 0 because a large fraction of gates can never be re-encoded, so “run to generation N ” is not a meaningful target—a fixed dose (or growth phase) is used instead. Defensively, the same fact means the encoding **resists DB-based simplification**, which is a property one wants of an obfuscation.

11 Implementation

- `src/replace/gadgets.rs`: `ProdConfig / ProdLedger` (`draw/inject/strip, fold_cg, emit_band_fill`); wired into `gadgetize_cnot`, `gadgetize_xgates`, and the `slice_zero_ccnot` wrappers. `ProdConfig::off` reproduces the plain gadget bit-identically.
- CLI: `sss --cnot --gadgetize --prod-k/-deg/-k-hi/-deg-hi/-band/-rsrc`; env-var control in `gen_sandwich_gadget`; `prod_grid.rs` harness.
- `hmap_affine`: `Reg::Triple` so `--degree 3` forms genuine triples over `--deg2-wire-list` (previously capped at pairs).
- Correctness: library suite 166/166; endpoint mismatches 0 at $n = 6, 16, 128$; masked builds pass the pinned zero-slice contract; no bare X .

12 Open threads

- The literal “generation 100” phase-A target is infeasible on the wide-gate gadget (§10); the pipeline uses a fixed DB dose + a $\times 3$ growth phase.
- A deg-4 tower needs scratch decomposition of width-8 fragments (the cubic borrowed-scratch pattern in `emit_poly_add` is the template).
- Quantify the mixing/DB-match cost of width-6 gates directly.
- Production review against non-reconstruction (static gate-statistics) distinguishers; the heatmaps are execution-based only.